

Name: Arava Raja

Reg No: 192365033

Course Code: CSA1718

PROGRAM – 9

Travelling Salesman

Aim

To implement the **Travelling Salesman Problem (TSP)** using Python and determine the **minimum cost path** that visits each city exactly once and returns to the starting city.

Algorithm

1. Read the number of cities and the cost (adjacency) matrix.
2. Select a starting city.
3. Create a list of all remaining cities except the starting city.
4. Generate all possible permutations (orders) of the remaining cities.
5. For each permutation:
6. Start from the selected city.
7. Calculate the total travel cost by visiting each city in the permutation.
8. Add the cost of returning to the starting city.
9. Compare the total cost with the current minimum cost.
10. If the current cost is smaller, update the minimum cost and store the corresponding path.
11. Repeat Steps 5–7 until all permutations are checked.

Python Program

```
from itertools import permutations
```

```
def travelling_salesman(graph, start):
```

```
    n = len(graph)
```

```
    vertices = [i for i in range(n) if i != start]
```

```
min_path = float('inf')

best_path = None

# Generate all possible permutations of cities

for perm in permutations(vertices):

    current_cost = 0

    current_city = start

    # Calculate the cost of the current route

    for city in perm:

        current_cost += graph[current_city][city]

        current_city = city

    # Return to the starting city

    current_cost += graph[current_city][start]

    # Update minimum cost and best path

    if current_cost < min_path:

        min_path = current_cost

        best_path = (start,) + perm + (start,)
```

```
    return min_path, best_path

# Example graph (Adjacency Matrix)

graph = [

    [0, 10, 15, 20],

    [10, 0, 35, 25],

    [15, 35, 0, 30],

    [20, 25, 30, 0]

]


start_city = 0


cost, path = travelling_salesman(graph, start_city)


print("Minimum Cost:", cost)

print("Optimal Path:", " -> ".join(map(str, path)))
```

Expected Output

Minimum Cost: 80

Optimal Path: 0 -> 1 -> 3 -> 2 -> 0

PROGRAM – 10

A* Algorithm

Aim

To implement the **A* Search Algorithm** in Python to find the shortest path between a start node and a goal node using heuristic values.

Algorithm

1. Create a graph with nodes and edge costs.
2. Assign a heuristic value $h(n)$ to each node.
3. Initialize the **Open List** with the start node.
4. Initialize an empty **Closed List**.
5. Select the node from the Open List with the lowest $f(n) = g(n) + h(n)$ value.
6. If the selected node is the goal node, stop and display the path.
7. Otherwise, move the selected node to the Closed List.
8. Expand all neighboring nodes.
9. Calculate the cost $g(n)$, heuristic $h(n)$, and total cost $f(n)$ for each neighbor.
10. Update the Open List if a better path is found.
11. Repeat Steps 5–10 until the goal node is reached or the Open List becomes empty.

Python Program

```
from heapq import heappush, heappop
```

```
# Graph represented as adjacency list
```

```
graph = {
```

```
'A': [('B', 1), ('C', 3)],
```

```
'B': [('D', 3), ('E', 6)],
```

```
'C': [('F', 5)],  
  
'D': [('G', 2)],  
  
'E': [('G', 1)],  
  
'F': [('G', 2)],  
  
'G': []  
  
}
```

```
# Heuristic values
```

```
heuristic = {  
  
    'A': 7,  
  
    'B': 6,  
  
    'C': 4,  
  
    'D': 2,  
  
    'E': 1,  
  
    'F': 2,  
  
    'G': 0  
  
}
```

```
def astar(start, goal):
```

```
    open_list = []
```

```
heappush(open_list, (heuristic[start], 0, start, [start]))

visited = set()

while open_list:

    f, g, node, path = heappop(open_list)

    if node == goal:

        return path, g

    if node in visited:

        continue

    visited.add(node)

    for neighbor, cost in graph[node]:

        if neighbor not in visited:

            new_g = g + cost

            new_f = new_g + heuristic[neighbor]

            heappush(open_list, (new_f, new_g, neighbor, path + [neighbor]))

return None, float('inf')
```

```
start = 'A'
```

```
goal = 'G'
```

```
path, cost = astar(start, goal)
```

```
print("Shortest Path:", " -> ".join(path))
```

```
print("Total Cost:", cost)
```

Expected Output

Shortest Path: A -> B -> D -> G

Total Cost: 6